

.NET 10 App Dev Hands-On Lab

DAL Lab 6 – Data Initialization

This lab walks you through creating the data initialization code. Before starting this lab, you must have completed DAL Lab 5.

Part 1: Create the Sample Data

- Create a new folder named Initialization in the AutoLot.Dal project. Add a file named SampleData.cs to the folder, and update the class to the following:

```
namespace AutoLot.Dal.Initialization;

internal static class SampleData
{
    public static List<Make> Makes =>
    [
        new() { Id = 1, Name = "VW" },
        new() { Id = 2, Name = "Ford" },
        new() { Id = 3, Name = "Saab" },
        new() { Id = 4, Name = "Yugo" },
        new() { Id = 5, Name = "BMW" },
        new() { Id = 6, Name = "Pinto" }
    ];
    public static List<Car> Inventory =>
    [
        new() { Id = 1, MakeId = 1, Color = "Black", PetName = "Zippy", Price = "50000" },
        new() { Id = 2, MakeId = 2, Color = "Rust", PetName = "Rusty", Price = "50000" },
        new() { Id = 3, MakeId = 3, Color = "Black", PetName = "Mel", Price = "50000" },
        new() { Id = 4, MakeId = 4, Color = "Yellow", PetName = "Clunker", Price = "50000" },
        new() { Id = 5, MakeId = 5, Color = "Black", PetName = "Bimmer", Price = "50000" },
        new() { Id = 6, MakeId = 5, Color = "Green", PetName = "Hank", Price = "50000" },
        new() { Id = 7, MakeId = 5, Color = "Pink", PetName = "Pinky", Price = "50000" },
        new() { Id = 8, MakeId = 6, Color = "Black", PetName = "Pete", Price = "50000" },
        new() { Id = 9, MakeId = 4, Color = "Brown", PetName = "Brownie", Price = "50000" },
        new() { Id = 10, MakeId=1, Color="Rust", PetName="Lemon", IsDrivable=false, Price="50000" }
    ];
    public static List<Radio> Radios =>
    [
        new() { Id= 1, CarId = 1, HasSubWoofers = true, RadioId = "SuperRadio 1", HasTweeters = true },
        new() { Id= 2, CarId = 2, HasSubWoofers = true, RadioId = "SuperRadio 2", HasTweeters = true },
        new() { Id= 3, CarId = 3, HasSubWoofers = true, RadioId = "SuperRadio 3", HasTweeters = true },
        new() { Id= 4, CarId = 4, HasSubWoofers = true, RadioId = "SuperRadio 4", HasTweeters = true },
        new() { Id= 5, CarId = 5, HasSubWoofers = true, RadioId = "SuperRadio 5", HasTweeters = true },
        new() { Id= 6, CarId = 6, HasSubWoofers = true, RadioId = "SuperRadio 6", HasTweeters = true },
        new() { Id= 7, CarId = 7, HasSubWoofers = true, RadioId = "SuperRadio 7", HasTweeters = true },
        new() { Id= 8, CarId = 8, HasSubWoofers = true, RadioId = "SuperRadio 8", HasTweeters = true },
        new() { Id= 9, CarId = 9, HasSubWoofers = true, RadioId = "SuperRadio 9", HasTweeters = true },
        new() { Id= 10, CarId=10, HasSubWoofers = true, RadioId = "SuperRadio 10", HasTweeters = true }
    ];
}
```

```

public static List<Driver> Drivers =>
[
    new() { Id = 1, PersonInformation = new() { FirstName = "Fred", LastName = "Flintstone" } },
    new() { Id = 2, PersonInformation = new() { FirstName = "Barney", LastName = "Rubble" } }
];
public static List<CarDriver> CarsAndDrivers =>
[
    new() { Id = 1, CarId = 1, DriverId = 1 },
    new() { Id = 2, CarId = 2, DriverId = 2 }
];
}

```

Part 2: Create the Store Data_INITIALIZER

- In the Initialization folder, create a new file named SampleDataInitializer.cs. Update the code to match the following:
- Change the class to public and static.

```

namespace AutoLot.Dal.Initialization;
public static class SampleDataInitializer
{
    internal static void ClearData(ApplicationDbContext context)
    {
        var entities = new[]
        {
            typeof(CarDriver).FullName, typeof(Driver).FullName,
            typeof(Radio).FullName, typeof(Car).FullName,
            typeof(Make).FullName, typeof(SeriLogEntry).FullName
        };
        IExecutionStrategy strategy = context.Database.CreateExecutionStrategy();
        strategy.Execute(() =>
        {
            using var transaction = context.Database.BeginTransaction();
            try
            {
                foreach (var entityName in entities)
                {
                    var entity = context.Model.FindEntityType(entityName);
                    var tableName = entity.GetTableName();
                    var schemaName = entity.GetSchema();
                    #pragma warning disable EF1002 // Risk of vulnerability to SQL injection.
                    context.Database.ExecuteSqlRaw($"DELETE FROM {schemaName}.{tableName}");
                    context.Database.ExecuteSqlRaw(
                        $"DBCC CHECKIDENT (\\"{schemaName}.{tableName}\\", RESEED, 1);");
                    #pragma warning restore EF1002 // Risk of vulnerability to SQL injection.
                }
                transaction.Commit();
            }
            catch
            {
                transaction.Rollback();
                throw;
            }
        });
    }
}

```

```

internal static void SeedData(ApplicationDbContext context)
{
    IExecutionStrategy strategy = context.Database.CreateExecutionStrategy();
    strategy.Execute(() =>
    {
        using var transaction = context.Database.BeginTransaction();
        try
        {
            ProcessInsert(context, context.Makes, SampleData.Makes);
            ProcessInsert(context, context.Drivers, SampleData.Drivers);
            ProcessInsert(context, context.Cars, SampleData.Inventory);
            ProcessInsert(context, context.Radios, SampleData.Radios);
            ProcessInsert(context, context.CarDrivers, SampleData.CarsAndDrivers);
            transaction.Commit();
        }
        catch
        {
            transaction.Rollback();
            throw;
        }
    });
    static void ProcessInsert<TEntity>(
        ApplicationDbContext context,
        DbSet<TEntity> table,
        List<TEntity> records) where TEntity : BaseEntity
    {
        if (table.Any())
        {
            return;
        }
        var metaData = context.Model.FindEntityType(typeof(TEntity).FullName);
        var identityInsertSql
            = $"SET IDENTITY_INSERT {metaData.GetSchema()}.{metaData.GetTableName()}";
        try
        {
#pragma warning disable EF1002 // Risk of vulnerability to SQL injection.
            context.Database.ExecuteSqlRaw($"{identityInsertSql} ON");
            table.AddRange(records);
            context.SaveChanges();
        }
        finally
        {
            // Ensure IDENTITY_INSERT is always turned off, even on failure.
            context.Database.ExecuteSqlRaw($"{identityInsertSql} OFF");
#pragma warning restore EF1002 // Risk of vulnerability to SQL injection.
        }
    }
}

public static void ClearAndReseedDatabase(ApplicationDbContext context)
{
    ClearData(context);
    SeedData(context);
}
}

```

Summary

This lab created a data initializer.

Next steps

The next lab is optional and adds integration tests to the data access layer.